

THE LIVING VAULT

From an Obsidian Knowledge Base to a Governed
Architecture of Agents and Autonomous M&A Intelligence

“A vault becomes intelligent not when an LLM can read it once, but when reasoning can enrich its memory, challenge its own conclusions, preserve evidence, and return to a better information set in the next cycle.”

Alejandro Reynoso

JUDGE BUSINESS SCHOOL.UNIVERSITY OF CAMBRIDGE.JULY 2026

Abstract

This paper develops a pedagogical architecture for combining a large language model with an Obsidian knowledge vault in investment banking. The laboratory begins with one thousand synthetic companies, each represented both as structured financial data and as a human-readable, linked Markdown note. It then advances through a five-level maturity ladder: a one-pass snapshot analyst; a scenario analyst operating across nine joint macro-credit and innovation states; a reflexive memory loop that reads, profiles, writes and rereads; a role-separated investment committee with an explicit red team; and a governed living system that detects events, updates state, records evidence and places consequential recommendations behind a human approval gate.

The principal objective is not to claim that an LLM can execute mergers and acquisitions. It is to show Master of Finance students how agency emerges from an architecture. The reasoning model, knowledge vault, retrieval engine, deterministic calculations, schemas, orchestrator, permissions, audit trail and stopping rules perform different functions. Obsidian supplies durable, inspectable institutional memory. The LLM supplies contextual synthesis. Code supplies reproducible calculation and controlled execution. Governance defines authority.

The implementation also produced an instructive failure. Early versions relied on large JSON inputs and unconstrained JSON-like outputs. NumPy types failed serialization; verbose model responses were truncated; handwritten parsing encountered incomplete strings. These were not cosmetic programming defects. They exposed the boundary between probabilistic intelligence and deterministic infrastructure. The revised design uses readable evidence packets, native schema-constrained outputs, validation, bounded response budgets, explicit timeouts and non-crashing diagnostic fallbacks. The paper therefore treats reliability engineering as part of agent design rather than as an appendix to it.

Keywords: autonomous agents; agentic workflows; Obsidian; knowledge vault; institutional memory; investment banking; mergers and acquisitions; scenario analysis; structured outputs; governance; auditability; human oversight.

Contents

1	Introduction: From a Repository to an Institutional Mind	4
2	The Teaching Laboratory	4
2.1	A deliberately synthetic investment-banking universe	4
2.2	Why Google Drive and Obsidian?	5
3	Five Components That Must Not Be Confused	5
4	The M&A Retrieval Engine	6
5	The Five-Level Maturity Ladder	6
5.1	Level 1: the snapshot analyst	6
5.2	Level 2: the scenario analyst	7
5.3	Level 3: reflexive memory	7
5.4	Level 4: the investment committee	8
5.5	Level 5: the governed living system	8
6	When Does the Workflow Become an Agent?	9
7	The Vault as Memory, Not as Truth	9
8	The JSON Failure as an Architectural Lesson	10
8.1	Failure one: data that looked like JSON was not JSON-safe	10
8.2	Failure two: asking for JSON did not guarantee JSON	10
8.3	Failure three: valid structured output was truncated	10
8.4	The repaired contract	10
9	Context Engineering and Evidence Discipline	11
10	Governance as the Architecture of Permission	11
10.1	A risk-tiered authority model	12
11	What the Student Should Learn	12
11.1	Financial learning objectives	12
11.2	AI architecture learning objectives	12
11.3	Governance learning objectives	12
12	From Colab Demonstration to Institutional System	12
13	Failure Modes Beyond JSON	13
13.1	Memory poisoning	13
13.2	Circular corroboration	13
13.3	Scenario theatre	13
13.4	Recommendation lock-in	13
13.5	Authority creep	13
13.6	False completion	13
13.7	Endless or expensive loops	13
14	Design Principles for the Living Vault	14

15 Conclusion: Intelligence That Returns to a Better Memory	14
A Minimal Pseudocode for the Governed Vault Loop	15
B Five-Level Implementation Checklist	15
C Glossary	15
References	16

1 Introduction: From a Repository to an Institutional Mind

There is a familiar rhythm in investment banking. Information arrives in fragments: a company profile, a management conversation, a market move, a refinancing need, a change in regulation, an acquisition rumour, a technological breakthrough. Analysts convert these fragments into models and presentations. Senior bankers add judgment. Committees decide which opportunities deserve attention. Yet the knowledge generated by one exercise is often dispersed among inboxes, spreadsheets, slide decks and individual memory. The institution may be highly intelligent while its memory remains surprisingly fragile.

Obsidian offers a deceptively simple alternative. It stores ordinary Markdown files in a folder called a vault. Notes can carry properties, links, tags, tables, citations and transclusions. Because the files remain transparent and portable, the vault can be read by people, searched by code and supplied to a language model. The importance of this arrangement is not aesthetic. It creates a common memory surface on which human and machine reasoning can meet.

An LLM connected to such a vault can perform a useful one-pass task: retrieve company notes, compare potential acquirers and targets, and draft an M&A memorandum. But the deeper architectural question begins after that first answer. Can the analysis itself become a new note? Can the next cycle reread the new information? Can alternative macroeconomic and innovation states reverse a recommendation? Can a red team challenge the thesis? Can the system remember its dissent, detect a changed fact and reopen the case? Can it do so without silently exceeding its authority?

Those questions take us from retrieval-augmented analysis to agents and, eventually, to bounded autonomy.

The central proposition

The vault is not an LLM, and the LLM is not the vault. The vault preserves durable institutional memory. The LLM interprets a selected context. Deterministic code calculates and retrieves. The orchestrator decides when each component is used. Governance defines what may be written, recommended, escalated or executed. Agency appears in the relations among these components.

The paper proceeds as a guided ascent. It first establishes the conceptual vocabulary, then constructs the synthetic financial universe, explains the five maturity levels, studies the shock architecture, examines the transition from workflow to agent, and ends with the governance and reliability lessons required for institutional deployment.

2 The Teaching Laboratory

2.1 A deliberately synthetic investment-banking universe

The Colab notebook creates a Drive-backed Obsidian vault containing 1,000 synthetic companies. Synthetic data serves three teaching purposes. First, it permits experimentation without confidentiality or market-abuse concerns. Second, it allows the instructor to embed economically meaningful relationships rather than merely draw independent random numbers. Third, a fixed seed makes the universe reproducible: students can debate reasoning over the same evidence.

Each company has an identifier, name, sector, subsector, geography, stage and ownership form. The financial layer includes revenue, growth, EBITDA, margin, leverage, cash conversion, research and development, capital expenditure, enterprise value, trading multiple and beta. The strategic layer includes capabilities, needs, moat, innovation, deal appetite, management signals, credit quality and ESG indicators. The notes are visibly labelled synthetic.

Table 1: The company note as a bridge between financial data and institutional narrative

Layer	Representative fields	Investment-banking use
Identity	Sector, subsector, region, country, stage, ownership	Peer selection, jurisdiction, buyer universe, ownership dialogue
Financial	Revenue, growth, EBITDA, margin, leverage, conversion, EV/EBITDA	Scale, affordability, financing capacity, valuation and accretion logic
Strategic	Capabilities, needs, moat, innovation, management signal	Industrial logic, adjacency, scarcity value and transaction appetite
Risk	Credit score, beta, ESG, capex intensity	Downside resilience, refinancing pressure and execution constraints
Memory	Links to profiles, shocks, sectors and deal notes	Provenance, path dependence, review and reuse

The master data is stored in CSV and Parquet form for calculation. The same companies are written as Markdown notes for human inspection. This dual representation is essential. Spreadsheets are efficient for vectorized ranking; notes are effective for narrative, links, provenance and incremental enrichment. The system does not force one representation to perform the other's function.

2.2 Why Google Drive and Obsidian?

The notebook mounts Google Drive and creates the vault beneath MyDrive. This is pedagogically convenient for Colab and operationally familiar to students. Obsidian then opens the synchronized folder as a vault. The resulting architecture is lightweight: Markdown remains the canonical human-readable memory, while tabular files support reproducible computation.

The important principle is portability. The vault is not a proprietary model memory that disappears when a chat ends. It is inspectable outside the model. A professor can open a note, trace a link, correct a claim and rerun the workflow. This reversibility is a foundation of governance.

3 Five Components That Must Not Be Confused

Contemporary language often collapses the model, agent, tool and data store into a single anthropomorphic actor. For finance students, that vocabulary is dangerous because it obscures accountability.

1. The **language model** maps context to a probabilistic output. In the laboratory, Claude Sonnet 5 performs contextual interpretation and synthesis.
2. The **vault** preserves durable state. It contains source facts, generated profiles, scenario notes, committee memoranda and audit artifacts.
3. The **retrieval and calculation layer** is deterministic code. It filters 1,000 companies, computes pair features and builds evidence packets.
4. The **orchestrator** sequences calls, supplies prompts, validates outputs, writes notes, detects changes and applies stopping rules.
5. The **governance layer** establishes permissions, human gates, audit records, timeouts, error handling and prohibited actions.

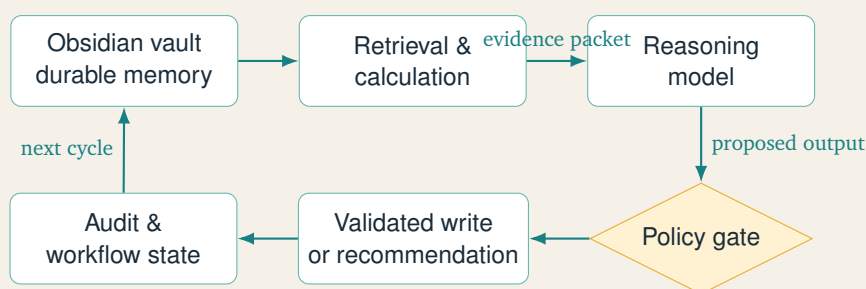


Figure 1: Agency is an architectural cycle, not a property of one box.

4 The M&A Retrieval Engine

Sending all 1,000 company notes to an LLM on every pass would be expensive, slow and analytically weak. More context is not synonymous with better context. The notebook therefore uses deterministic retrieval to create a plausible opportunity set before invoking the model.

For a potential buyer b and target t , the heuristic score combines same-sector and adjacent-sector logic, need-capability complementarity, geographic expansion, feasible relative size, buyer financing capacity, innovation fit, management appetite and valuation distance:

$$S_{bt} = 18I_{\text{same}} + 10I_{\text{adj}} + 24F_{\text{need/cap}} + 9G + 14Z + 12C + 8N + 5A - 8V.$$

The coefficients are transparent teaching weights, not estimated causal parameters. Their role is to reduce a combinatorial universe to a set worth examining. A strong classroom discussion should ask what the score omits: antitrust, tax, accounting treatment, shareholder reaction, country risk, integration history, management credibility and the availability of superior strategic alternatives.

The retrieval layer identifies up to 40 candidates while limiting concentration in any single buyer or target. Claude receives a smaller evidence packet, not an unfiltered data dump.

A crucial distinction

The retrieval score answers, “Which pairs deserve analytical attention?” It does not answer, “Which transaction should be executed?” Deterministic screening, LLM judgment, valuation, diligence and authorization are separate stages.

5 The Five-Level Maturity Ladder

The maturity ladder is the conceptual centre of the laboratory. Each level changes the relationship between reasoning and memory.

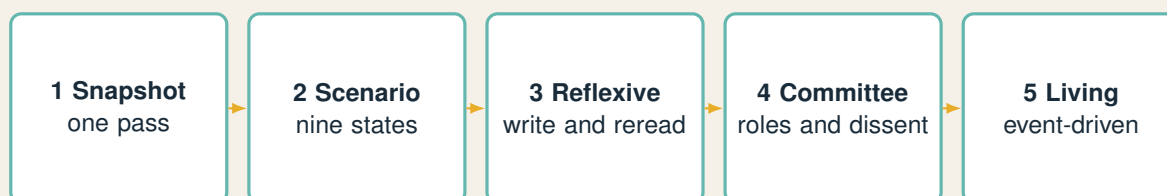


Figure 2: The ladder moves from an answer over memory to a governed process that changes memory.

5.1 Level 1: the snapshot analyst

At Level 1, Obsidian is a source repository and a destination for one memorandum. The retrieval engine selects candidate pairs. The LLM evaluates strategic logic, revenue and cost synergies, financing capacity, principal risks and diligence questions. The memorandum is written to the

vault.

This is useful augmented analysis, but it is not yet strongly agentic. The model receives an assembled context and produces one answer. It does not revisit its conclusion after observing a consequence. Its relation to memory is linear:

vault $\rightarrow$ retrieve $\rightarrow$ reason $\rightarrow$ memo.

The teaching value is foundational. Students see that an LLM need not replace the financial model. Code calculates; the LLM interprets. They also learn that the evidence packet is a design choice. A model cannot cite evidence it was never shown.

5.2 Level 2: the scenario analyst

Level 2 adds external states. Three macro-credit shocks interact with three innovation shocks to create nine joint scenarios. The model no longer asks only which deal looks attractive under the baseline. It asks which thesis remains coherent across alternative worlds.

The three credit states are:

- M1. Credit expansion:** falling rates, tighter spreads and greater debt capacity.
- M2. Orderly tightening:** higher rates and spreads, but continued access for strong credits.
- M3. Credit seizure:** sharply wider spreads and impaired leveraged-finance issuance.

The three innovation states are:

- I1. AI productivity wave:** rewards proprietary data, software and automation.
- I2. Disruptive substitution:** compresses incumbent margins and impairs obsolete assets.
- I3. Regulated transition:** slows deployment but raises the value of licenses, compliance capacity, resilience and scale.

Table 2: The nine-state scenario matrix

	I1: Productivity	I2: Substitution	I3: Regulated transition
M1: Expansion	Cheap finance amplifies capability acquisitions	High multiples may conceal obsolescence	Licensed scale commands scarcity value
M2: Tightening	Synergy must outrun financing cost	Selective distress creates opportunity	Strong credits gain bargaining power
M3: Seizure	Strategic value survives only with liquidity	Impairment and refinancing interact	Regulatory assets may defend downside

For each pair and state, code adjusts financing resilience, target growth and enterprise value. A robustness measure rewards mean and worst-case performance while penalizing dispersion:

$$R_{bt} = 0.60\bar{r}_{bt} + 0.35 \min_s r_{bt,s} - 0.05\sigma(r_{bt,s}).$$

Again, this is an explicit classroom functional, not an empirical estimator. Its purpose is to make robust decision logic inspectable.

5.3 Level 3: reflexive memory

Level 3 closes the first genuine memory loop:

read $\rightarrow$ profile $\rightarrow$ write $\rightarrow$ reread $\rightarrow$ revise.

Claude creates evidence-bounded profiles for the companies involved in the leading transactions. Each profile may contain a strategic archetype, transferable capabilities, gaps, likely

buyer-or-target role, financing constraints, shock sensitivities, hidden optionality, red flags and unresolved questions. The profile is written to the company's linked vault note. A second pass then receives both original facts and the newly created profile.

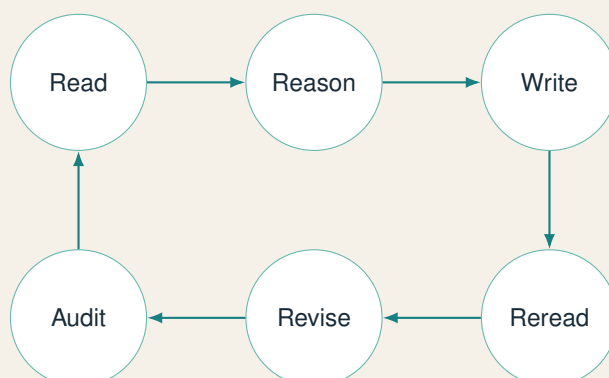


Figure 3: The reflexive loop changes the future information set.

This is path dependence. The output of cycle k becomes part of the state observed in cycle $k + 1$. The analytical history matters. Yet enrichment also creates a new risk: a weak inference can become durable memory and later be mistaken for fact. The note must therefore distinguish reported data, deterministic calculation, scenario assumption and LLM inference.

5.4 Level 4: the investment committee

Level 4 introduces role separation. One model performs several bounded passes, each with a different mandate:

- the **industrial strategist** tests products, customers, geography and capability fit;
- the **financing lead** tests valuation, leverage, liquidity and downside funding;
- the **integration lead** tests systems, talent, culture and synergy realization;
- the **red team** tries to falsify the thesis;
- the **chair** reconciles the evidence without erasing dissent.

These are not five conscious executives. They are role-separated reasoning passes. Their advantage is procedural: a financing objection cannot disappear merely because the industrial narrative is eloquent. The chair's memo preserves a dissent register, conditions precedent, stage gates and an explicit statement of non-authorization.

Why the red team matters

In finance, an attractive narrative is not evidence of a robust transaction. The red team changes the objective from “make the deal persuasive” to “discover what would make the thesis false.” This is an evaluator-optimizer pattern: proposal, criticism, revision and bounded decision.

5.5 Level 5: the governed living system

Level 5 replaces a manually initiated analysis with an event-conditioned cycle. The system fingerprints its data and current shock state. If no material change has occurred, it records a skip and stops. If the state has changed, it retrieves candidates, analyzes the active scenario, calls a challenger, obtains a chair decision, writes a versioned memo and places any proposed advancement in a human approval queue.

The critical word is *governed*. The notebook remains recommendation-only. It cannot contact

management, transmit confidential data, instruct a trade, sign an engagement letter, commit capital or execute a transaction. Autonomy applies to bounded analytical work, not to institutional authority.

6 When Does the Workflow Become an Agent?

The term *agent* is often applied too broadly. A useful continuum is:

Table 3: Workflow, agent and autonomous agent

Property	Workflow	Agent	Autonomous agent
Path	Predetermined sequence	Model selects among admissible next actions	Selection repeats over time and events
State	Passed between steps	Used to revise plan	Durable across cycles
Tool choice	Coded in advance	Context-dependent	Context- and policy-dependent
Stopping	End of script	Goal or rule based	Goal, event, budget, risk and human gates
Authority	Fixed execution	Bounded delegation	Persistent but revocable delegation
Example	Level 1 pipeline	Level 3/4 loop	Level 5 governed monitor

Levels 1 and 2 are best understood as increasingly sophisticated workflows. Level 3 exhibits agentic properties because a prior inference changes the subsequent context. Level 4 adds deliberative organization. Level 5 approaches operational autonomy because it can decide to act, wait, stop or escalate when state changes. Nevertheless, the autonomy is local and bounded.

A compact state transition description is:

$$x_{k+1} = U(x_k, o_k, a_k), \quad a_k = \pi(g, x_k, o_k, \mathcal{T}, \mathcal{P}),$$

where g is the goal, x_k the stored state, o_k the current observation, $\mathcal{T}$ the admissible tools and $\mathcal{P}$ the policy constraints. The update function U writes validated results to memory. The policy π may select analysis, retrieval, write, wait, escalation or termination.

7 The Vault as Memory, Not as Truth

Obsidian provides durable memory, but durability does not guarantee correctness. A mature vault requires epistemic labels.

1. **Reported fact:** directly present in an identified source.
2. **Derived calculation:** reproducible from cited inputs and a visible formula.
3. **Scenario assumption:** deliberately hypothetical and state-dependent.
4. **Analyst inference:** a human judgment supported by evidence.
5. **Model inference:** an LLM-generated judgment requiring validation.
6. **Disputed claim:** a proposition with conflicting evidence.

The laboratory links generated profiles to original company notes rather than overwriting source fields. This separation preserves lineage. A future implementation should add note-level hashes, source dates, model identifiers, prompt versions, reviewer identity and supersession links.

Memory creates both intelligence and liability

Without memory, an agent repeats itself. With ungoverned memory, it can compound an error. The design problem is therefore not whether to remember, but what to remember, with what status, for how long, and subject to whose review.

8 The JSON Failure as an Architectural Lesson

The initial notebook failed in ways that are particularly valuable for teaching autonomous systems.

8.1 Failure one: data that looked like JSON was not JSON-safe

Pandas and NumPy use scalar types such as `numpy.float64`. They display like ordinary numbers but are not always accepted by Python's default JSON encoder. A large nested evidence packet could therefore fail before the model was called. This was a boundary error between the analytical data layer and the communication layer.

8.2 Failure two: asking for JSON did not guarantee JSON

A prompt such as “return valid JSON” is an instruction to a probabilistic generator, not a formal contract. The model may add prose, code fences, missing fields or an unexpected type. Handwritten extraction of the first opening brace and final closing brace is fragile.

8.3 Failure three: valid structured output was truncated

The revised system used native schema-constrained output, but the Level 4 red team and chair still produced incomplete strings. The error reported end-of-file while parsing. The schema was not the fundamental problem: the response had been cut off. Claude Sonnet 5 uses adaptive thinking by default, and thinking tokens count against the same maximum-output budget as the visible answer. A large evidence packet, high effort, verbose schema and automatic doubled retry created long latency and truncation.

A parser needs a complete contract. The fragment `{"status":"draft", "analysis":"The transaction... is not an imperfect memo; it is no memo at all. An autonomous architecture must treat truncation as a failed transaction, not as a partially trustworthy recommendation.`

8.4 The repaired contract

The final design introduced seven controls:

1. convert NumPy and Pandas values into ordinary Python scalars;
2. render model inputs as compact Markdown evidence packets rather than giant JSON;
3. use the SDK's native structured-output interface and Pydantic validation;
4. impose concise schemas and explicit limits on findings, risks and actions;
5. disable adaptive thinking for the bounded Level 4 reporting calls;
6. apply one visible attempt and a 120-second timeout rather than an indefinite doubled retry;
7. return a readable diagnostic object on failure so the notebook continues safely.

The broader lesson is that agent reliability depends on **interfaces**. Intelligence may be probabilistic, but the handoff to code must be typed, bounded, validated and observable. Structured outputs constrain form; they do not remove refusals, truncation, authorization errors or bad

evidence. Reliability is architectural.

9 Context Engineering and Evidence Discipline

An LLM's context window is finite even when it is very large. More importantly, attention is an economic resource. Flooding the model with every vault file increases cost and may dilute the facts that matter. The notebook therefore follows a context-engineering sequence:

universe → deterministic filter → evidence packet → role-specific context → bounded output.

At Level 4, the chair does not require every raw company record. It needs the champion pair, the relevant company facts, the scenario states, and the four specialist conclusions. This is hierarchical compression. The challenge is to compress without hiding dissent or laundering uncertain claims into facts.

Evidence identifiers offer a partial solution. Every material finding should cite a company or pair ID. A production design would validate that each cited identifier exists and that claims are entailed by the referenced record. Groundedness, coverage and source quality should be evaluated separately.

10 Governance as the Architecture of Permission

Finance is not merely another domain for autonomous work. It combines confidential information, fiduciary duties, market conduct rules, model risk, reputational consequences and legal authorization. A system that can recommend a transaction must not thereby acquire permission to conduct it.

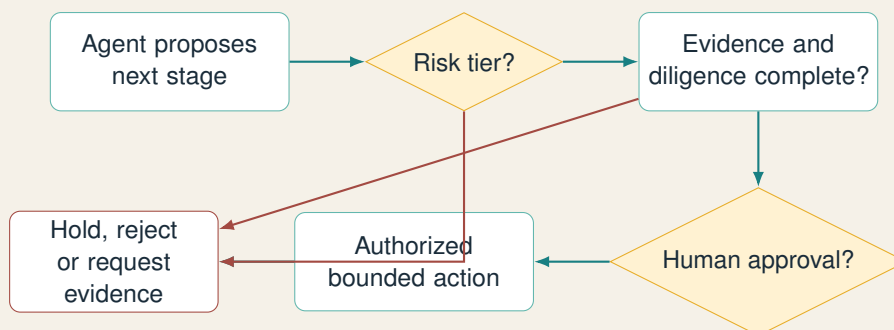


Figure 4: The recommendation crosses gates; authority does not emerge from confidence.

The Level 5 audit record includes a timestamp, event type, payload hash, model name and dry-run status. The state file records the most recent signature and decision. The approval queue requires an identifiable human decision. These controls map naturally to the NIST AI Risk Management Framework functions: govern, map, measure and manage.

10.1 A risk-tiered authority model

Table 4: Illustrative authority tiers for the M&A vault

Tier	Examples	Agent authority	Required control
Low	Read notes, calculate ratios, generate synthetic profiles	Automatic	Logging and validation
Moderate	Write an analytical note, update a scenario, rank opportunities	Automatic within sandbox	Provenance, versioning, reviewer visibility
High	Share a memo, contact a counterparty, alter a live pipeline	Propose only	Named human approval and confidentiality check
Prohibited	Trade, commit capital, sign, misrepresent, use inside information	None	Technical prevention and escalation

11 What the Student Should Learn

The notebook is designed for Master of Finance students, not merely for programmers. Its learning objectives are architectural and financial.

11.1 Financial learning objectives

Students should be able to distinguish strategic fit from affordability; understand why financing conditions can reverse industrial logic; separate average scenario performance from worst-case resilience; formulate kill conditions; and recognize that integration risk is part of valuation rather than an operational afterthought.

11.2 AI architecture learning objectives

Students should be able to distinguish a model from an agent; explain why memory creates path dependence; describe the role of deterministic retrieval; design an evaluator-optimizer loop; identify where a schema contract is required; and place a human gate at the boundary between analysis and consequential action.

11.3 Governance learning objectives

Students should be able to classify facts and inferences, trace evidence, interpret an audit record, design stopping conditions, diagnose false completion and determine which tasks may be automated, proposed or prohibited.

Suggested classroom exercise

Divide the class into five groups corresponding to the Level 4 roles. Give every group the same pair and nine-state matrix. Require each group to write a 300-word memo citing only supplied IDs. The chair must then issue a decision that preserves at least one material dissent. Finally, ask the class which new note should be written to the vault and whether it is a fact, calculation, assumption or inference.

12 From Colab Demonstration to Institutional System

The notebook is a transparent prototype. A production system would require a more formal architecture.

1. **Identity and access management:** least-privilege service accounts, scoped vault access and separation between research, confidential and public information.
2. **Data contracts:** versioned schemas, validation at ingestion and explicit treatment of missing or stale fields.
3. **Provenance:** source-level citations, immutable raw records, note lineage and supersession.
4. **Evaluation:** groundedness, ranking stability, scenario sensitivity, calibration, latency, cost and human override rates.
5. **Model risk management:** approved uses, benchmark cases, change control, independent validation and rollback.
6. **Transaction controls:** idempotency, approval tokens, dual control, confidentiality barriers and a permanent separation between analysis and execution.
7. **Monitoring:** drift, repeated failures, abnormal tool use, memory poisoning and unexpected changes in recommendation.

MCP or another standardized tool boundary could later expose controlled vault search, note retrieval, calculation and writing capabilities. MCP does not itself supply judgment or permission. It standardizes how an AI application discovers and invokes resources and tools. The host still owns security boundaries, credentials and policy.

13 Failure Modes Beyond JSON

13.1 Memory poisoning

A speculative profile may be reread as authoritative truth. Mitigation requires epistemic labels, source links, review status and expiry.

13.2 Circular corroboration

The model may cite a note that was generated from its own earlier conclusion. Provenance graphs must distinguish independent evidence from derived restatement.

13.3 Scenario theatre

Nine scenarios can create an illusion of rigour if the transmission functions are arbitrary. Weights should be debated, sensitivity-tested and, where possible, estimated.

13.4 Recommendation lock-in

Once a deal becomes the champion, subsequent agents may anchor on it. The red team should receive permission to propose a superior alternative or reject the opportunity set.

13.5 Authority creep

A notebook that begins by writing notes may later be connected to email, CRM or execution systems. Every new tool changes the risk perimeter and requires explicit authorization.

13.6 False completion

A syntactically valid memo may omit decisive evidence. Schema validity is not semantic validity. Completion must include coverage, groundedness and unresolved-item tests.

13.7 Endless or expensive loops

An evaluator can continually request refinement. Cycles require call budgets, timeouts, materiality thresholds and termination rules.

14 Design Principles for the Living Vault

1. Preserve the distinction between memory and truth.
2. Retrieve before reasoning; do not confuse a larger prompt with better evidence.
3. Let deterministic code calculate and probabilistic models interpret.
4. Write generated intelligence beside source facts, never invisibly over them.
5. Make dissent durable.
6. Treat schemas, timeouts and fallbacks as components of agency.
7. Design stopping conditions before enabling repetition.
8. Grant authority by action class, not by model confidence.
9. Record what the system observed, proposed, executed and refused.
10. Keep the human gate closest to the consequential action.

15 Conclusion: Intelligence That Returns to a Better Memory

The laboratory begins with a modest proposition: connect an LLM to an Obsidian vault containing financial information. It ends with a much richer architecture. A static repository becomes a scenario laboratory; the laboratory becomes reflexive memory; reflexive memory becomes a committee; the committee becomes a governed process that can wait, stop, reopen and escalate.

The progression clarifies what an agent is. It is not simply a fluent model, a long prompt, a tool call or a repeated script. It is a system in which a goal, state, observations, admissible actions, feedback and stopping conditions are coordinated over time. An autonomous agent adds persistence and delegated initiative, but only within a defined authority envelope.

For investment banking, the most important contribution is not automatic deal execution. It is the possibility of an institutional memory that is continuously enriched yet remains inspectable; an analytical process that can preserve both thesis and dissent; and a governance structure that makes each consequential boundary visible.

The JSON failures reinforce the central lesson. The quality of an autonomous system is not equal to the intelligence of its model. It is constrained by the weakest interface through which reasoning becomes state, action or memory. A sophisticated model coupled to a brittle parser is a brittle system. A probabilistic model surrounded by typed contracts, deterministic calculation, bounded context, evidence discipline, auditability and human authority can become a trustworthy participant in financial work.

The final proposition

The living vault is not a second brain because it contains many notes. It becomes a form of institutional intelligence when every cycle can recover evidence, reason within a role, write a qualified result, preserve disagreement, observe change and return to a memory that is better than the one with which it began.

A Minimal Pseudocode for the Governed Vault Loop

```

while within_budget():
    observation = detect_material_change(vault, shocks)
    if not observation.changed:
        audit("no_material_change")
        stop()

    candidates = deterministic_retrieval(vault, observation)
    evidence = build_bounded_evidence_packet(candidates)

    thesis = structured_llm_call(role="analyst", evidence=evidence)
    challenge = structured_llm_call(
        role="red_team", evidence=evidence, prior=thesis
    )
    decision = structured_llm_call(
        role="chair", evidence=evidence,
        prior=[thesis, challenge]
    )

    validate_schema(decision)
    validate_evidence_ids(decision, vault)
    write_versioned_memo(vault, decision)
    audit("decision_written", hash(decision))

    if decision.requires_consequential_action:
        queue_for_human_approval(decision)
        stop()

```

B Five-Level Implementation Checklist

Level	Memory relation	Required controls	Completion evidence
1	Read once; write one memo	Retrieval manifest, evidence IDs, schema validation	Ranked memo and limitations
2	Read once under nine states	Shock definitions, deterministic transmission, robustness calculation	Scenario ranking, reversals and kill conditions
3	Read, enrich, write and reread	Epistemic labels, provenance, versioning	Profile notes and revised thesis
4	Shared evidence, specialist passes, chair synthesis	Role mandates, bounded outputs, dissent register, timeout	Specialist memos, chair decision and conditions
5	Persistent event-driven state	Fingerprints, budgets, audit ledger, human gate, prohibited actions	Versioned cycle memo and approval queue

C Glossary

Agent	A system in which a model selects among admissible actions using a goal, state, observation and feedback.
Autonomous agent	An agent with persistent, delegated initiative across time or events, subject to stopping and authority rules.

Knowledge vault	A durable, inspectable collection of linked notes and data files used as institutional memory.
Orchestrator	Code that assembles context, invokes models or tools, validates results, updates state and controls repetition.
Structured output	A response generated under a formal schema and validated before use.
Reflexive memory	A loop in which generated analysis is written to memory and changes a later reasoning context.
Human gate	A control that prevents a consequential proposed action from proceeding without named human authorization.

References

- [1] Anthropic (2024). “Building Effective Agents.” Anthropic Engineering. <https://www.anthropic.com/engineering/building-effective-agents>.
- [2] Anthropic (2025). “Effective Context Engineering for AI Agents.” Anthropic Engineering. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>.
- [3] Anthropic (2025). “How We Built Our Multi-Agent Research System.” Anthropic Engineering. <https://www.anthropic.com/engineering/multi-agent-research-system>.
- [4] Anthropic (2026). “Structured Outputs.” Claude Platform Documentation. <https://platform.claude.com/docs/en/build-with-claude/structured-outputs>.
- [5] Anthropic (2026). “Adaptive Thinking.” Claude Platform Documentation. <https://platform.claude.com/docs/en/build-with-claude/adaptive-thinking>.
- [6] Model Context Protocol (2025). “Specification and Architecture.” <https://modelcontextprotocol.io/specification/2025-11-25>.
- [7] National Institute of Standards and Technology (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST AI 100-1. <https://doi.org/10.6028/NIST.AI.100-1>.
- [8] National Institute of Standards and Technology (2024). *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. NIST AI 600-1. <https://doi.org/10.6028/NIST.AI.600-1>.
- [9] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. and Cao, Y. (2023). “ReAct: Synergizing Reasoning and Acting in Language Models.” *International Conference on Learning Representations*.
- [10] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K. and Yao, S. (2023). “Reflexion: Language Agents with Verbal Reinforcement Learning.” *Advances in Neural Information Processing Systems*, 36.
- [11] Park, J. S., O’Brien, J., Cai, C. J., Morris, M. R., Liang, P. and Bernstein, M. S. (2023). “Generative Agents: Interactive Simulacra of Human Behavior.” *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*.
- [12] Obsidian (2026). “Obsidian Help: Vaults, Properties, Links and Files.” <https://help.obsidian.md/>.